

Ampliación Django: roles y permisos

Objetivo

Añadir un sistema de permisos para que:

- un usuario normal solo pueda gestionar sus propios productos;
 - un administrador pueda ver, editar y eliminar cualquier producto;
 - determinadas vistas estén restringidas según el tipo de usuario.
-

1. Aprovechar los permisos de Django

Django ya incluye un sistema de permisos mediante:

- `is_authenticated`
- `is_staff`
- `is_superuser`
- grupos;
- permisos específicos.

Para esta práctica usaremos inicialmente:

```
request.user.is_staff
```

Un usuario con:

```
is_staff = True
```

será considerado administrador de la aplicación.

2. Lógica de permisos

La regla será:

Usuario normal

↓

Solo puede editar sus propios productos

Mientras que:

Administrador



Puede editar cualquier producto

3. Modificar la vista de edición

Antes teníamos:

```
producto = get_object_or_404(
    Producto,
    pk=pk,
    usuario=request.user
)
```

Ahora necesitamos permitir acceso también al administrador.

Podemos hacerlo así:

```
@login_required
def producto_editar(request, pk):

    producto = get_object_or_404(
        Producto,
        pk=pk
    )

    if (
        producto.usuario != request.user
        and not request.user.is_staff
    ):
        raise PermissionDenied

    if request.method == "POST":

        form = ProductoForm(
            request.POST,
            instance=producto
        )

        if form.is_valid():

            form.save()

            messages.success(
                request,
                "Producto actualizado correctamente."
```

```

    )

    return redirect(
        "producto_lista"
    )

else:

    form = ProductoForm(
        instance=producto
    )

    return render(
        request,
        "productos/formulario.html",
        {
            "form": form,
            "titulo": "Editar producto",
        }
    )

```

Debemos importar:

```
from django.core.exceptions import PermissionDenied
```

4. Modificar la eliminación

```

@login_required
def producto_eliminar(request, pk):

    producto = get_object_or_404(
        Producto,
        pk=pk
    )

    if (
        producto.usuario != request.user
        and not request.user.is_staff
    ):
        raise PermissionDenied

    if request.method == "POST":

        producto.delete()

        messages.success(

```

```
        request,
        "Producto eliminado correctamente."
    )

    return redirect(
        "producto_lista"
    )

    return render(
        request,
        "productos/eliminar.html",
        {
            "producto": producto
        }
    )
```

5. Mostrar botones según permisos

En los templates podemos comprobar:

```
{% if user == producto.usuario or user.is_staff %}
```

Por ejemplo:

```
{% if user == producto.usuario or user.is_staff %}
```

```
<a
    href="{% url 'producto_editar' producto.pk %}"
    class="btn btn-outline-secondary btn-sm"
>
    Editar
</a>

<a
    href="{% url 'producto_eliminar' producto.pk %}"
    class="btn btn-outline-danger btn-sm"
>
    Eliminar
</a>
```

```
{% endif %}
```

Esto hace que el administrador vea los botones de edición y eliminación para todos los productos.

6. Crear panel de administración interno

Podemos crear una vista exclusiva para usuarios `staff`.

```
from django.contrib.admin.views.decorators import staff_member_required
```

Y añadir:

```
@staff_member_required
def panel_admin(request):

    productos = Producto.objects.select_related(
        "categoria",
        "usuario"
    ).order_by(
        "-fecha_creacion"
    )

    usuarios = User.objects.all().order_by(
        "username"
    )

    context = {
        "productos": productos,
        "usuarios": usuarios,
    }

    return render(
        request,
        "administracion/panel.html",
        context
    )
```

7. Importar `User`

En `views.py`:

```
from django.contrib.auth.models import User
```

8. Crear URL del panel

En:

productos/urls.py

Añadir:

```
path(
    "panel-admin/",
    views.panel_admin,
    name="panel_admin"
),
```

9. Crear template del panel

Crear:

templates/administracion/panel.html

Código:

```
{% extends "base.html" %}

{% block title %}
Panel de administración
{% endblock %}

{% block content %}

<h1 class="mb-4">
    Panel de administración
</h1>

<div class="row mb-4">

    <div class="col-md-6">

        <div class="card shadow-sm">

            <div class="card-body">

                <h5>
                    Usuarios
                </h5>

                <p class="display-6">
                    {{ usuarios.count }}
                </p>

            </div>

        </div>

    </div>

</div>
```

</div>

</div>

</div>

<div class="col-md-6">

<div class="card shadow-sm">

<div class="card-body">

<h5>

Productos

</h5>

<p class="display-6">

{{ productos.count }}

</p>

</div>

</div>

</div>

</div>

<div class="card shadow-sm">

<div class="card-body">

<h2 class="h4 mb-3">

Todos los productos

</h2>

<div class="table-responsive">

<table class="table table-hover">

<thead>

<tr>

<th>

Producto

</th>

```
<th>
    Usuario
</th>
```

```
<th>
    Categoría
</th>
```

```
<th>
    Precio
</th>
```

```
<th>
    Acciones
</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
{% for producto in productos %}
```

```
<tr>
```

```
<td>
    {{ producto.nombre }}
</td>
```

```
<td>
    {{ producto.usuario.username }}
</td>
```

```
<td>
    {{ producto.categoria.nombre }}
</td>
```

```
<td>
    {{ producto.precio }} €
</td>
```

```
<td>
```

```
<a
    href="{% url 'producto_editar' producto.pk %}"
    class="btn btn-sm btn-outline-secondary"
>
```

```

        Editar
    </a>

    <a
        href="{% url 'producto_eliminar' producto.pk %}"
        class="btn btn-sm btn-outline-danger"
    >
        Eliminar
    </a>

</td>

</tr>

{% empty %}

<tr>

<td colspan="5">

        No existen productos.

</td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

</div>

</div>

{% endblock %}

```

10. Añadir enlace al panel

En `base.html`:

```
{% if user.is_staff %}
```

```
<li class="nav-item">

    <a
      class="nav-link"
      href="{% url 'panel_admin' %}"
    >
      Administración
    </a>

</li>

{% endif %}
```

Un usuario normal no verá este enlace.

11. Crear usuarios administradores

La forma más sencilla es desde:

/admin/

Entrar con un superusuario.

Después:

Usuarios
↓
Seleccionar usuario
↓
Activar
"Es staff"

También se puede crear directamente:

```
python manage.py createsuperuser
```

12. Diferencia entre staff y superusuario

is_staff

Permite identificar usuarios administrativos.

`user.is_staff`

Puede usarse para acceder al panel interno.

is_superuser

Indica que el usuario tiene todos los permisos de Django.

`user.is_superuser`

Un superusuario normalmente también tiene:

`is_staff = True`

13. Mostrar rol en la interfaz

Podemos mostrar:

```
{% if user.is_staff %}

    <span class="badge bg-danger">
        Administrador
    </span>

{% else %}

    <span class="badge bg-secondary">
        Usuario
    </span>

{% endif %}
```

Por ejemplo:

Hola, juan Usuario

o:

Hola, admin Administrador

14. Usar grupos de Django

Para un proyecto más avanzado es mejor utilizar grupos.

Por ejemplo:

Administradores

Editores

Usuarios

Django dispone del modelo:

Group

Importar:

```
from django.contrib.auth.models import Group
```

15. Crear un grupo

Desde Django Admin:

Administración



Grupos



Añadir grupo

Crear:

Editores

Después asignarle permisos.

16. Comprobar si un usuario pertenece a un grupo

Podemos comprobar:

```
request.user.groups.filter(  
    name="Editores"  
)exists()
```

Ejemplo:

```
es_editor = request.user.groups.filter(
    name="Editores"
).exists()
```

17. Crear función auxiliar

Podemos crear:

```
def es_editor(user):

    return user.groups.filter(
        name="Editores"
    ).exists()
```

Después:

```
if es_editor(request.user):
    ...
```

18. Permisos personalizados

Django crea automáticamente permisos para cada modelo:

```
add_producto
change_producto
delete_producto
view_producto
```

Podemos comprobar:

```
request.user.has_perm(
    "productos.change_producto"
)
```

19. Usar decorator de permisos

También existe:

```
from django.contrib.auth.decorators import permission_required
```

Ejemplo:

```
@permission_required(
    "productos.delete_producto",
    raise_exception=True
)
def producto_eliminar_admin(request, pk):
```

...

Si el usuario no dispone del permiso:

403 Forbidden

20. Modelo de permisos recomendado

Podemos establecer tres niveles:

USUARIO NORMAL



Crear productos
Editar productos propios
Eliminar productos propios
Ver todos los productos
EDITOR



Todo lo anterior
Editar cualquier producto
ADMINISTRADOR



Todo lo anterior
Eliminar cualquier producto
Gestionar usuarios
Gestionar categorías

21. Ejemplo de función de permiso

Podemos centralizar la lógica:

```
def puede_editar_producto(
    user,
    producto
):
    if user.is_staff:
```

```
return True
```

```
return producto.usuario == user
```

Y usar:

```
if not puede_editar_producto(  
    request.user,  
    producto  
):  
    raise PermissionDenied
```

Esto mejora la organización.

22. Crear archivo **permissions.py**

Crear:

productos/permissions.py

Código:

```
def puede_editar_producto(  
    user,  
    producto  
):  
  
    if not user.is_authenticated:  
        return False  
  
    if user.is_staff:  
        return True  
  
    return producto.usuario == user  
  
def puede_eliminar_producto(  
    user,  
    producto  
):  
  
    if not user.is_authenticated:  
        return False  
  
    if user.is_staff:
```

```
return True
```

```
return producto.usuario == user
```

23. Usarlo desde las vistas

Importar:

```
from .permissions import (  
    puede_editar_producto,  
    puede_eliminar_producto,  
)
```

Después:

```
if not puede_editar_producto(  
    request.user,  
    producto  
):  
    raise PermissionDenied
```

24. Seguridad importante

No debemos confiar solo en esto:

```
{% if user.is_staff %}
```

Los templates solo controlan lo que se muestra.

La seguridad real debe estar también en la vista.

Incorrecto:

Ocultar botón
pero permitir acceso directo por URL

Correcto:

Ocultar botón
+
Comprobar permiso en servidor

25. Flujo del usuario normal

Login
↓
Crear producto
↓
Producto.usuario = request.user
↓
Mis productos
↓
Editar propio
↓
Eliminar propio

26. Flujo del administrador

Login
↓
Panel administración
↓
Ver todos los productos
↓
Editar cualquier producto
↓
Eliminar cualquier producto

27. Pruebas recomendadas

Crear:

juan
maria
admin

Configurar:

juan
is_staff = False

maria
is_staff = False

admin

is_staff = True

Prueba 1

Juan crea:

Portátil

Juan puede:

Ver

Editar

Eliminar

Prueba 2

María puede ver:

Portátil

pero no debe poder:

Editar

Eliminar

Prueba 3

Admin puede ver:

Portátil

y también:

Editar

Eliminar

aunque el propietario sea Juan.

28. Resultado final del proyecto

Con todas las ampliaciones realizadas, el proyecto ya incluye:

Django

- Modelos relacionados
- CRUD
- Registro
- Login
- Logout
- Recuperación de contraseña
- Email con token
- Nueva contraseña
- Propiedad de productos
- Autenticación
- Autorización
- Roles
- Permisos
- Panel administrativo

El proyecto pasa así de ser una práctica básica de CRUD a una aplicación mucho más completa para trabajar seguridad, usuarios, propiedad y permisos en Django.